



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“Semana 13 — Introducción a Árboles Binarios de Búsqueda”

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

Fase 1: COMPRENDE	3
Fase 2: APLICA	3
Fase 3: REFLEXIONA	4
Fase 4: VALIDA	4

Fase 1: COMPRENDE

COMPRENDE-1: ¿Cuál es la invariante del BST?

Para todo nodo n: todas las claves del subárbol izquierdo son menores que n.clave,
y todas las claves del subárbol derecho son mayores que n.clave.

COMPRENDE-2: ¿Por qué el in-order de un BST produce orden ascendente?

Porque el recorrido in-order visita Izquierdo -> Nodo -> Derecho (LNR) y, por la invariante del BST,
primero salen todas las claves menores, luego la clave del nodo, y al final las mayores.

COMPRENDE-3: ¿Cuándo el BST supera al dict y cuándo el dict supera al BST?

dict supera al BST en búsquedas exactas por clave (promedio $O(1)$) cuando no necesitas orden.
BST supera al dict cuando necesitas recorrido ordenado o consultas por rango (ej. puntajes entre 8.0 y 9.0).

IA-REFLEXION-C:

Aprendimos que el orden del BST depende de comparar claves: menores van a la izquierda y mayores a la derecha,
y que respetar esa regla (invariante) es lo que hace posible buscar por rango y recorrer ordenado.

Fase 2: APLICA

VALIDA (T1-T4)

arbol = construir_bst(CATALOGO_S13)

T1: inorder igual que sorted()

claves_io = [c for c, v in arbol.inorder()]

claves_ok = sorted(p for p, t, g in CATALOGO_S13)

print("T1:", claves_io == claves_ok)

T2: buscar exacto

assert arbol.buscar(9.1) == ("Origen", "Sci-Fi")

assert arbol.buscar(5.0) is None

assert arbol.buscar(6.5) == ("Avatar", "Accion")

print("T2: OK")

T3: árbol vacío

av = BST()

```

assert av.buscar(7.0) is None
assert av.inorder() == []
assert av.buscar_rango(6.0, 9.0) == []
assert av.altura() == 0
print("T3: OK")

# T4: rango [8.0, 9.0]
en_rango = arbol.buscar_rango(8.0, 9.0)
claves_r = sorted(c for c, v in en_rango)
claves_e = sorted(p for p, t, g in CATALOGO_S13 if 8.0 <= p <= 9.0)
print("T4:", claves_r == claves_e)

# VALIDA: T1=___ T2=___ T3=___ T4=___ T5=___

```

Fase 3: REFLEXIONA

IA-REFLEXION-A:

```

# El método más difícil fue buscar_rango porque no es un recorrido "fijo"
como inorder:
# uno decide cuándo bajar a la izquierda o derecha y cuándo podar ramas
completas según minv y maxv.
# Si no podas ramas, se vuelve parecido a recorrer todo el árbol (O(n)),
perdiendo la ventaja del BST.

```

Fase 4: VALIDA

"""

streamMX_bst_s13.py

Semana 13 - Introduccion a Arboles Binarios de Busqueda

Equipo: [NOMBRE DEL EQUIPO]

Integrantes: [LISTA]

Fecha: [FECHA]

```

# COMPRENDE-1: Para todo nodo n: subárbol izq < n.clave y
subárbol der > n.clave.
# COMPRENDE-2: In-order (LNR) da orden ascendente por la
invariante: izq (menores), nodo, der (mayores).
# COMPRENDE-3: dict gana en búsqueda exacta O(1) promedio
sin orden; BST gana en orden y rangos.
# IA-REFLEXION-C: Aprendimos que la invariante del BST permite
ordenar y podar ramas en búsquedas por rango.
"""

```

import time, random

class Nodo:

```
def __init__(self, clave, valor):  
    self.clave = clave  
    self.valor = valor  
    self.izq = None  
    self.der = None
```

```
class BST:
```

```
    def __init__(self):  
        self.raiz = None
```

```
    def insert(self, clave, valor):  
        self.raiz = self._insert_rec(self.raiz, clave, valor)
```

```
    def _insert_rec(self, nodo, clave, valor):  
        if nodo is None:  
            return Nodo(clave, valor)  
  
        if clave < nodo.clave:  
            nodo.izq = self._insert_rec(nodo.izq, clave, valor)  
        elif clave > nodo.clave:  
            nodo.der = self._insert_rec(nodo.der, clave, valor)  
        else:  
            nodo.valor = valor # sobrescribe si hay duplicado  
  
        return nodo # CRÍTICO: siempre regresar el nodo
```

```
    def buscar(self, clave):  
        return self._buscar_rec(self.raiz, clave)
```

```
    def _buscar_rec(self, nodo, clave):  
        if nodo is None:  
            return None  
        if clave == nodo.clave:  
            return nodo.valor  
        elif clave < nodo.clave:  
            return self._buscar_rec(nodo.izq, clave)  
        else:  
            return self._buscar_rec(nodo.der, clave)
```

```
    def inorder(self):  
        resultado = []  
        self._inorder_rec(self.raiz, resultado)  
        return resultado
```

```
    def _inorder_rec(self, nodo, resultado):  
        if nodo is None:  
            return
```

```

        self._inorder_rec(nodo.izq, resultado)
        resultado.append((nodo.clave, nodo.valor))
        self._inorder_rec(nodo.der, resultado)

def preorder(self):
    resultado = []
    self._preorder_rec(self.raiz, resultado)
    return resultado

def _preorder_rec(self, nodo, resultado):
    if nodo is None:
        return
    resultado.append((nodo.clave, nodo.valor))
    self._preorder_rec(nodo.izq, resultado)
    self._preorder_rec(nodo.der, resultado)

def postorder(self):
    resultado = []
    self._postorder_rec(self.raiz, resultado)
    return resultado

def _postorder_rec(self, nodo, resultado):
    if nodo is None:
        return
    self._postorder_rec(nodo.izq, resultado)
    self._postorder_rec(nodo.der, resultado)
    resultado.append((nodo.clave, nodo.valor))

def buscar_rango(self, min_clave, max_clave):
    resultado = []
    self._rango_rec(self.raiz, min_clave, max_clave, resultado)
    return resultado

def _rango_rec(self, nodo, minv, maxv, resultado):
    if nodo is None:
        return

    if nodo.clave > minv:
        self._rango_rec(nodo.izq, minv, maxv, resultado)

    if minv <= nodo.clave <= maxv:
        resultado.append((nodo.clave, nodo.valor))

    if nodo.clave < maxv:
        self._rango_rec(nodo.der, minv, maxv, resultado)

def altura(self):

```

```
    return self._altura_rec(self.raiz)
```

```
def _altura_rec(self, nodo):  
    if nodo is None:  
        return 0  
    alt_izq = self._altura_rec(nodo.izq)  
    alt_der = self._altura_rec(nodo.der)  
    return 1 + max(alt_izq, alt_der)
```

```
CATALOGO_S13 = [  
    (8.2, "Interestelar",      "Sci-Fi"),  
    (7.5, "El Laberinto del Fauno", "Fantasia"),  
    (9.1, "Origen",           "Sci-Fi"),  
    (6.8, "Dune Parte 1",      "Sci-Fi"),  
    (8.7, "Oppenheimer",       "Drama"),  
    (7.5, "La Senal",          "Thriller"),  
    (8.0, "Parasite",          "Drama"),  
    (7.1, "Arrival",           "Sci-Fi"),  
    (9.0, "Schindler's List",   "Drama"),  
    (8.4, "The Dark Knight",    "Accion"),  
    (6.5, "Avatar",            "Accion"),  
    (7.8, "Tar",                "Drama"),  
    (8.9, "Spirited Away",      "Animacion"),  
    (7.3, "Blade Runner 2049",  "Sci-Fi"),  
    (9.3, "El Senor de los Anillos", "Fantasia"),  
]
```

```
def imprimir_arbol_2D(nodo, nivel=0, prefijo="Raiz: "):  
    if nodo is not None:  
        imprimir_arbol_2D(nodo.der, nivel + 1, "Der: ")  
        print(" " * (nivel * 6) + prefijo + str(nodo.clave))  
        imprimir_arbol_2D(nodo.izq, nivel + 1, "Izq: ")
```

```
def construir_bst(catalogo):  
    arbol = BST()  
    for puntaje, titulo, genero in catalogo:  
        arbol.insert(puntaje, (titulo, genero))  
    return arbol
```

```
# ---- RETO 10 -- BENCHMARK ----
```

```
def benchmark_rango(n, minv=7.0, maxv=8.5, reps=500):  
    datos = [(round(random.uniform(6.0, 10.0), 2), f"P_{i}", "G")  
              for i in range(n)]
```

```
    bst = BST()
```

```

for p, t, g in datos:
    bst.insert(p, (t, g))

cat_dict = {p: (t, g) for p, t, g in datos}
lista_ord = sorted(datos, key=lambda x: x[0])

def rango_dict(d, minv, maxv):
    return [(k, v) for k, v in d.items() if minv <= k <= maxv]

def rango_lista(lst, minv, maxv):
    return [(p, (t, g)) for p, t, g in lst if minv <= p <= maxv]

t0 = time.perf_counter()
for _ in range(reps): bst.buscar_rango(minv, maxv)
t_bst = (time.perf_counter()-t0)/reps*1000

t0 = time.perf_counter()
for _ in range(reps): rango_dict(cat_dict, minv, maxv)
t_dict = (time.perf_counter()-t0)/reps*1000

t0 = time.perf_counter()
for _ in range(reps): rango_lista(lista_ord, minv, maxv)
t_lista = (time.perf_counter()-t0)/reps*1000

    print(f"n={n:>7}   BST={t_bst:.4f}ms   dict={t_dict:.4f}ms
lista={t_lista:.4f}ms")
    if t_bst > 0:
        print(f"                                dict/BST={t_dict/t_bst:.1f}x
lista/BST={t_lista/t_bst:.1f}x")

def benchmark_degeneracion(n):
    datos_aleatorios = [(float(i + random.uniform(0, 0.001)),
f"P_{i}", "G")
        for i in random.sample(range(n), n)]
    bst_ok = BST()
    for p, t, g in datos_aleatorios:
        bst_ok.insert(p, (t, g))

    datos_ordenados = [(float(i), f"P_{i}", "G") for i in range(n)]
    bst_deg = BST()
    for p, t, g in datos_ordenados:
        bst_deg.insert(p, (t, g))

reps = 200
minv, maxv = n * 0.3, n * 0.7

```



```

t0 = time.perf_counter()
for _ in range(reps): bst_ok.buscar_rango(minv, maxv)
t_ok = (time.perf_counter() - t0) / reps * 1000

t0 = time.perf_counter()
for _ in range(reps): bst_deg.buscar_rango(minv, maxv)
t_deg = (time.perf_counter() - t0) / reps * 1000

h_ok = bst_ok.altura()
h_deg = bst_deg.altura()

print(f"n={n:>6} aleatorio: {t_ok:.4f}ms (h={h_ok:>4}) "
      f"ordenado: {t_deg:.4f}ms (h={h_deg:>5}) "
      f"ratio={t_deg/t_ok:.1f}x" if t_ok > 0 else "t_ok=0")

if __name__ == "__main__":
    print("=" * 60)
    print("STREAMMX BST -- Semana 13")
    print("=" * 60)

    arbol = construir_bst(CATALOGO_S13)
    print(f"[INFO] Arbol construido con {len(CATALOGO_S13)}
nodos")
    print(f"[INFO] Altura del arbol: {arbol.altura()}")

    print("\n-- Estructura del arbol (debug visual) --")
    imprimir_arbol_2D(arbol.raiz)

    print("\n-- Recorrido In-Order (ascendente) --")
    for clave, valor in arbol.inorder():
        print(f" {clave:.1f} -> {valor[0]}")

    print("\n-- Busqueda exacta --")
    print(f" buscar(8.7) -> {arbol.buscar(8.7)}")
    print(f" buscar(5.0) -> {arbol.buscar(5.0)}")

    print("\n-- Busqueda por rango [8.0, 9.0] --")
    for clave, valor in arbol.buscar_rango(8.0, 9.0):
        print(f" {clave:.1f} -> {valor[0]}")

    # IA-REFLEXION-A:
    # El método más difícil fue buscar_rango porque requiere
    decidir cuándo podar ramas según el rango.

    print("\n" + "="*60)
    print("BENCHMARK -- Busqueda por rango [7.0, 8.5]")

```

```

print("="*60)
for n in [100, 1_000, 5_000, 10_000]:
    benchmark_rango(n)

print("\n" + "="*60)
print("BENCHMARK DEGENERACION: BST aleatorio vs BST
ordenado")
print("="*60)
for n in [100, 500, 1000, 2000]:
    benchmark_degeneracion(n)

```

```

"""
streamMX_bst_s13.py
Semana 13 - Introduccion a Arboles Binarios de Busqueda
Equipo: [NOMBRE DEL EQUIPO]
Integrantes: [LISTA]
Fecha: [FECHA]

# COMPRENDE-1: Cual es la invariante del BST? (1-2 oraciones)
#todos los valores izquierdos son menores al nodo principal y los
valores derechos son mayores a dicho nodo

# COMPRENDE-2: Por que el in-order de un BST produce orden
ascendente?
#por que todo subarbol izquierd contiene valores menores al nodo y
viceversa con el subarbol derecho

# COMPRENDE-3: Cuando el BST supera al dict y cuando el dict
supera al BST?
#cuando tienes que buscar, insertar o borrar suele ser mejorel
dict pero si es que ocupas ordenamiento suele ser mejor el arbol

# IA-REFLEXION-C: [Que aprendio el equipo que no sabia antes de la
IA]
#que el nodo principal suele ser como el ombligo o la raiz del
arbol y que los elementos dependiendo de su tamaño se clasifican
en la izquierda o derecha
"""

import time, random

# ---- CLASE NODO -- Pre-implementada. No modificar. ----
class Nodo:

```

```

"""
clave : valor de ordenamiento (puntaje float en StreamMX)
valor : datos asociados (titulo, genero)
izq    : hijo izquierdo (Nodo o None)
der    : hijo derecho   (Nodo o None)
"""

def __init__(self, clave, valor):
    self.clave = clave
    self.valor = valor
    self.izq   = None
    self.der   = None

# ---- CLASE BST -- El equipo implementa los metodos TODO ----
class BST:
    def __init__(self):
        self.raiz = None

    def insert(self, clave, valor):
        """Inserta un nodo. O(h)."""
        self.raiz = self._insert_rec(self.raiz, clave, valor)

    def _insert_rec(self, nodo, clave, valor):
        """
        TODO: Insercion recursiva.
        Casos:
        1. nodo es None -> crear y retornar Nodo(clave, valor)
        2. clave < nodo.clave -> insertar en subarbol izquierdo
        3. clave > nodo.clave -> insertar en subarbol derecho
        4. clave == nodo.clave -> actualizar valor (no duplicar
nodo)

        Siempre retornar el nodo al final.
        """
        #1)
        if nodo is None:
            return Nodo(clave, valor)

        #2) y 3)

        if clave < nodo.clave:
            nodo.izq = self._insert_rec(nodo.izq, clave, valor)
        elif clave > nodo.clave:

```

```

        nodo.der = self._insert_rec(nodo.der, clave, valor)
    else:
        #4)
        nodo.valor = valor
        return nodo

def buscar(self, clave):
    """Busca por clave exacta. Retorna valor o None. O(h)."""
    return self._buscar_rec(self.raiz, clave)

def _buscar_rec(self, nodo, clave):
    """
    TODO: Búsqueda recursiva.
    Casos:
    1. nodo es None -> retornar None
    2. clave == nodo.clave -> retornar nodo.valor
    3. clave < nodo.clave -> buscar en izquierdo
    4. clave > nodo.clave -> buscar en derecho
    """
    if nodo is None:
        return None
    if clave == nodo.clave:
        return nodo.valor
    elif clave < nodo.clave:
        return self._buscar_rec(nodo.izq, clave)
    else:
        return self._buscar_rec(nodo.der, clave)

def inorder(self):
    """Retorna lista de (clave, valor) en orden ascendente.
    O(n)."""
    resultado = []
    self._inorder_rec(self.raiz, resultado)
    return resultado

def _inorder_rec(self, nodo, resultado):
    """TODO: Orden LNR -> izquierdo, nodo, derecho."""
    if nodo is None:
        return
    self._inorder_rec(nodo.izq, resultado)

```

```

        resultado.append((nodo.clave, nodo.valor))
        self._inorder_rec(nodo.der, resultado)

def preorder(self):
    resultado = []
    self._preorder_rec(self.raiz, resultado)
    return resultado

def _preorder_rec(self, nodo, resultado):
    """TODO: Orden NLR -> nodo, izquierdo, derecho."""
    resultado.append((nodo.clave, nodo.valor))
    self._preorder_rec(nodo.izq, resultado)
    self._preorder_rec(nodo.der, resultado)

def postorder(self):
    resultado = []
    self._postorder_rec(self.raiz, resultado)
    return resultado

def _postorder_rec(self, nodo, resultado):
    """TODO: Orden LRN -> izquierdo, derecho, nodo."""
    self._postorder_rec(nodo.izq, resultado)
    self._postorder_rec(nodo.der, resultado)
    resultado.append((nodo.clave, nodo.valor))

def buscar_rango(self, min_clave, max_clave):
    """Retorna lista (clave, valor) con min <= clave <= max.
    O(log n + k)."""
    resultado = []
    self._rango_rec(self.raiz, min_clave, max_clave,
resultado)
    return resultado

```

```

def _rango_rec(self, nodo, minv, maxv, resultado):
    """
    TODO: Búsqueda por rango recursiva.
    Pistas:
        - Si nodo es None: retornar (caso base)
        - Si nodo.clave > minv: explorar subarbol izquierdo
        - Si minv <= nodo.clave <= maxv: agregar a resultado
        - Si nodo.clave < maxv: explorar subarbol derecho
    La eficiencia viene de descartar ramas completas.
    """

    if nodo is None:
        return
    if nodo.clave > minv:
        self._rango_rec(nodo.izq, minv, maxv, resultado)

        if minv <= nodo.clave <= maxv:
            resultado.append((nodo.clave, nodo.valor))

        if nodo.clave < maxv:
            self._rango_rec(nodo.der, minv, maxv, resultado)


def altura(self):
    """Retorna la altura del arbol. O(n)."""
    return self._altura_rec(self.raiz)


def _altura_rec(self, nodo):
    """
    TODO: Altura recursiva.
    Arbol vacio = 0, solo raiz = 1
    General = 1 + max(altura_izq, altura_der)
    """

    if nodo is None:
        return 0
    alt_izq = self._altura_rec(nodo.izq)
    alt_der = self._altura_rec(nodo.der)
    return 1 + max(alt_izq, alt_der)

```

```

# ---- CATALOGO STREAMMX ----
CATALOGO_S13 = [
    (8.2, "Interestelar", "Sci-Fi"),
    (7.5, "El Laberinto del Fauno", "Fantasia"),
    (9.1, "Origen", "Sci-Fi"),
    (6.8, "Dune Parte 1", "Sci-Fi"),
    (8.7, "Oppenheimer", "Drama"),
    (7.5, "La Senal", "Thriller"),
    (8.0, "Parasite", "Drama"),
    (7.1, "Arrival", "Sci-Fi"),
    (9.0, "Schindler's List", "Drama"),
    (8.4, "The Dark Knight", "Accion"),
    (6.5, "Avatar", "Accion"),
    (7.8, "Tar", "Drama"),
    (8.9, "Spirited Away", "Animacion"),
    (7.3, "Blade Runner 2049", "Sci-Fi"),
    (9.3, "El Senor de los Anillos", "Fantasia"),
]

def imprimir_arbol_2D(nodo, nivel=0, prefijo="Raiz: "):
    """
    Herramienta de debug -- Pre-implementada. No modificar.
    Imprime el arbol en consola con indentacion que refleja
    la estructura real del arbol (derecho arriba, izquierdo
    abajo).

    Util para verificar que insert construyo el arbol
    correctamente
    antes de confiar en inorder().

    Ejemplo de salida para {50, 30, 70, 20, 40}:
        Der: 70.0
    Raiz: 50.0
        Izq: 30.0
            Der: 40.0
            Izq: 20.0
    """
    if nodo is not None:
        imprimir_arbol_2D(nodo.der, nivel + 1, "Der: ")
        print(" " * (nivel * 6) + prefijo + str(nodo.clave))

```

```

        imprimir_arbol_2D(nodo.izq, nivel + 1, "Izq: ")

def construir_bst(catalogo):
    arbol = BST()
    for puntaje, titulo, genero in catalogo:
        arbol.insert(puntaje, (titulo, genero))
    return arbol

if __name__ == "__main__":
    print("=" * 60)
    print("STREAMMX BST -- Semana 13")
    print("=" * 60)
    arbol = construir_bst(CATALOGO_S13)
    print(f"[INFO] Arbol construido con {len(CATALOGO_S13)}
nodos")
    print(f"[INFO] Altura del arbol: {arbol.altura()}")

    print("\n-- Estructura del arbol (debug visual) --")
    print("(Derecho arriba, Izquierdo abajo -- usa esto si inorder
falla)")
    imprimir_arbol_2D(arbol.raiz)

    print("\n-- Recorrido In-Order (ascendente) --")
    for clave, valor in arbol.inorder():
        print(f" {clave:.1f} -> {valor[0]}")

    print("\n-- Busqueda exacta --")
    print(f" buscar(8.7) -> {arbol.buscar(8.7)}")
    print(f" buscar(5.0) -> {arbol.buscar(5.0)}")

    print("\n-- Busqueda por rango [8.0, 9.0] --")
    for clave, valor in arbol.buscar_rango(8.0, 9.0):
        print(f" {clave:.1f} -> {valor[0]}")

# IA-REFLEXION-A: [El equipo escribe que metodo le resulto mas
difícil y por que]
#diria que el mas complicado fue el busqueda por rango ya que no
sigue un recorrido por asi decir, sino que tu eres el que debe de
decidir cuando bajar o descartar subarboles

```



```

# ---- RETO 10 -- BENCHMARK ----
def benchmark_rango(n, minv=7.0, maxv=8.5, reps=500):
    datos = [(round(random.uniform(6.0, 10.0), 2), f"P_{i}", "G")
              for i in range(n)]

    bst = BST()
    for p, t, g in datos:
        bst.insert(p, (t, g))

    cat_dict = {p: (t, g) for p, t, g in datos}
    lista_ord = sorted(datos, key=lambda x: x[0])

    def rango_dict(d, minv, maxv):
        """TODO: iterar d.items() y filtrar clave en [minv,
maxv]."""
        return [(k,v) for k, v in d.items() if minv <= k <= maxv]

    def rango_lista(lst, minv, maxv):
        """TODO: iterar lista_ord y filtrar. O(n)."""
        return [(p, (t, g)) for p, t, g in lst if minv <= p
<=maxv]

    t0 = time.perf_counter()
    for _ in range(reps): bst.buscar_rango(minv, maxv)
    t_bst = (time.perf_counter()-t0)/reps*1000

    t0 = time.perf_counter()
    for _ in range(reps): rango_dict(cat_dict, minv, maxv)
    t_dict = (time.perf_counter()-t0)/reps*1000

    t0 = time.perf_counter()
    for _ in range(reps): rango_lista(lista_ord, minv, maxv)
    t_lista = (time.perf_counter()-t0)/reps*1000

    print(f"n={n:>7}      BST={t_bst:.4f}ms      dict={t_dict:.4f}ms
lista={t_lista:.4f}ms")
    if t_bst > 0:
        print(f"                  dict/BST={t_dict/t_bst:.1f}x
lista/BST={t_lista/t_bst:.1f}x")

```

```

print(" " + "="*60)
print("BENCHMARK -- Búsqueda por rango [7.0, 8.5]")
print("="*60)
for n in [100, 1_000, 5_000, 10_000]:
    benchmark_rango(n)

# MID-CANONICO: [registrar tiempos obtenidos en su maquina]
# n=100:   BST=___ms  dict=___ms  lista=___ms
# n=1000:  BST=___ms  dict=___ms  lista=___ms
# n=5000:  BST=___ms  dict=___ms  lista=___ms
# n=10000: BST=___ms  dict=___ms  lista=___ms
# DECISION: [A partir de que n el BST es claramente mejor que el dict?]
#
#           [Existe algun n donde el dict supera al BST? Por que?]

# ---- BENCHMARK SECUNDARIO: arbol degenerado ----
# Compara BST con datos ALEATORIOS vs BST con datos ORDENADOS
# para ver en vivo como la degeneracion colapsa el rendimiento.
def benchmark_degeneracion(n):
    # BST con datos aleatorios (caso promedio -- balanceado)
    datos_aleatorios = [(float(i + random.uniform(0, 0.001)),
f"P_{i}", "G")
                        for i in random.sample(range(n), n)]

    bst_ok = BST()
    for p, t, g in datos_aleatorios:
        bst_ok.insert(p, (t, g))

    # BST con datos ORDENADOS (peor caso -- degenerado)
    datos_ordenados = [(float(i), f"P_{i}", "G") for i in
range(n)]
    bst_deg = BST()
    for p, t, g in datos_ordenados:
        bst_deg.insert(p, (t, g))

    reps = 200
    minv, maxv = n * 0.3, n * 0.7 # buscar el 40% central

    t0 = time.perf_counter()
    for _ in range(reps): bst_ok.buscar_rango(minv, maxv)

```

```

t_ok = (time.perf_counter() - t0) / reps * 1000

t0 = time.perf_counter()
for _ in range(reps): bst_deg.buscar_rango(minv, maxv)
t_deg = (time.perf_counter() - t0) / reps * 1000

h_ok = bst_ok.altura()
h_deg = bst_deg.altura()

print(f"n={n:>6}   aleatorio: {t_ok:.4f}ms (h={h_ok:>4})   "
      f"ordenado: {t_deg:.4f}ms (h={h_deg:>5})   "
      f"ratio={t_deg/t_ok:.1f}x" if t_ok > 0 else "t_ok=0")

print("\n" + "="*60)
print("BENCHMARK DEGENERACION: BST aleatorio vs BST ordenado")
print("(mismo buscar_rango -- diferente altura del arbol)")
print("="*60)
for n in [100, 500, 1000, 2000]:
    benchmark_degeneracion(n)

# MID-CANONICO-DEG: [registrar tiempos y alturas del benchmark
secundario]
#n=    100   aleatorio: 0.0003ms (h=    0)   ordenado: 0.0003ms (h=
0)   ratio=0.9x
#n=    500   aleatorio: 0.0003ms (h=    0)   ordenado: 0.0003ms (h=
0)   ratio=0.9x
#n=   1000   aleatorio: 0.0003ms (h=    0)   ordenado: 0.0003ms (h=
0)   ratio=1.0x
#n=   2000   aleatorio: 0.0003ms (h=    0)   ordenado: 0.0003ms (h=
0)   ratio=0.9x
# DECISION-DEG: [Confirma el dato: con n=1000, el arbol degenerado
es 100
#
mas lento. Este ratio corresponde a la diferencia
entre
#
O(log n) y O(n): log2(1000) ~ 10, vs 1000 pasos.]

```